Literature Review

1. Purpose and Scope

This review explains the concepts behind a real-time IoT pipeline, compares the main tool options for each layer, and gives the reason for each choice.

2. Background

2.1 IoT data and why streaming

IoT devices produce continuous, time-stamped readings. Value is highest when readings are acted on quickly (equipment faults, environment alarms), so batch loads that run hourly or daily are often too slow. Sensor data is also messy as devices send duplicates, readings arrive late or out of order, and values can be missing or physically impossible. A pipeline has to handle these cases by design.

2.2 Architecture styles

- **Lambda architecture** runs a batch layer and a speed layer side by side. It gives accurate and fast results but means two code bases to maintain.
- **Kappa architecture** treats everything as a stream over a replayable log. It has one code base, and reprocessing means replaying the log (Kreps, 2014).
- **Medallion layering** (raw, clean, aggregated) organizes stored data by quality level. It fits the project's layered storage and makes replay and audit easy.

The project uses an event-driven, Kappa-style design with layered storage. Kafka is the replayable log, and a single streaming job produces the clean and aggregated layers.

2.3 Core streaming concepts

Concept	Meaning	Why it matters here
Event time vs processing time	When a reading happened vs when the system saw it	Windows must use event time or late data lands in the wrong window (Akidau et al., 2015)
Watermark	The system's estimate of how late data can still arrive	Decides when a window is final and when old state can be dropped
Windowing	Grouping events into time buckets (tumbling, sliding)	Produces per-minute and per-site aggregates for the dashboard

Concept	Meaning	Why it matters here
Delivery semantics	At-most-once, at-least-once, exactly-once	At-least-once delivery plus idempotent writes can give an exactly-once effect, depending on the full persistence and recovery design
Deduplication	Removing repeated events using a unique event ID	Sensors and retries create duplicates
Dead-letter queue	A separate store for records that fail validation	Keeps bad data out of analytics without losing it
Backpressure and consumer lag	A consumer falling behind the producer	An early sign of an overloaded pipeline; a KPI in this project

2.4 Data quality and observability

Streaming data needs checks at ingestion: schema validation, range checks, completeness and freshness. Observability adds metrics on latency, throughput, lag and error rates so that problems are seen before users notice. The project builds both into the pipeline instead of adding them later. Data quality rules are written and reported with Great Expectations.

3. Related Work: Tool Comparison

Each table compares the chosen tool with common alternatives. Statements are general characteristics, and the last column gives the reason for the decision.

3.1 Message broker

Option	Strengths	Limitations for this project	Decision
Apache Kafka	Durable partitioned log, replay within the retention period, high throughput, large ecosystem, works with Spark	Heavier to run than a simple broker	**Chosen:** replay supports reprocessing and the Kappa design; widely used in industry
RabbitMQ	Flexible routing, mature queueing	Classic queues remove messages once acknowledged, so replay is limited (RabbitMQ Streams add a log model)	Not chosen

Option	Strengths	Limitations for this project	Decision
MQTT broker (for example Mosquitto)	Very light, the standard protocol for constrained devices	Not built for long retention or stream processing at scale	Not chosen; could sit in front of Kafka in a real deployment
Azure Event Hubs / IoT Hub	Managed, scalable, Kafka-compatible endpoint (Event Hubs)	Paid cloud service, outside the free-tools rule	Not chosen; design maps to it directly

3.2 Stream processing

Option	Strengths	Limitations for this project	Decision
PySpark Structured Streaming	Python API, SQL-like operations, event-time windows and watermarks, same engine as the course (Armbrust et al., 2018)	Micro-batch model gives latency of seconds, not milliseconds	**Chosen:** fits the track, the team's skills, and the latency target
Apache Flink	Event-at-a-time processing, low latency, strong state handling (Carbone et al., 2015)	Steeper operations and learning curve, not covered in the course	Not chosen
Kafka Streams	Library inside the application, no separate cluster	JVM languages only, so no Python	Not chosen
Azure Stream Analytics	Managed, SQL-like queries	Paid, cloud only	Not chosen

3.3 Storage and serving

Option	Strengths	Limitations for this project	Decision
Apache Parquet	Columnar, compressed, open file format, good for raw and clean history and replay	No transactions or table features by itself; many small files need periodic compaction	**Chosen (confirmed)** for the raw and clean layers; the lake_compaction DAG handles small

Option	Strengths	Limitations for this project	Decision
			files
SQL Server Developer	Relational model with keys and constraints, T-SQL, SSMS tooling, in the course scope, free for development and testing	Developer edition is not licensed for production; heavier than lightweight databases	**Chosen (confirmed)** for the serving layer; supports the ER diagram and logical and physical schema in the guidelines
Delta Lake	Adds ACID transactions, schema enforcement and time travel on top of Parquet	Extra dependency and complexity	Not chosen; the lake format is plain Apache Parquet
PostgreSQL	Free open-source relational database	Outside the confirmed stack	Not chosen
TimescaleDB or InfluxDB	Purpose-built for time series	Extra component and different design work	Not chosen
Azure Data Lake or Synapse	Industry-grade cloud analytics	Paid, cloud only	Not chosen

3.4 Dashboard

Option	Strengths	Limitations for this project	Decision
Grafana	Free, strong time-series panels, auto-refresh, built-in Microsoft SQL Server data source, runs in Docker	Another service to configure	**Chosen (confirmed)**
Streamlit	Pure Python, quick to build	Less polished for time-series monitoring	Not chosen
Power BI	Familiar in the Microsoft ecosystem	Desktop is free, but live streaming and sharing need paid features	Not chosen

Decision: Grafana, approved on 17 Oct 2026. Streamlit was considered and dropped.

3.5 Data quality

Option	Strengths	Limitations for this project	Decision
Great Expectations	Open-source Python framework with reusable expectation suites, stored validation results and generated data docs; works with Spark and SQL data sources	Built around batch validation, so it validates the stored clean Parquet and SQL Server tables while Spark keeps the per-event checks; setup effort	**Chosen (confirmed)**
Deequ	Library for data unit tests on Spark	Scala-first, Python access through PyDeequ	Not chosen
Soda Core	Open-source checks written in a YAML-style language	Another tool outside the course	Not chosen
Custom rules and a Python runner	Full control and small footprint	Reporting, documentation and upkeep are built by hand	Not chosen

3.6 Orchestration and deployment

Option	Role	Decision
Apache Airflow	Schedules the five required DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill and warehouse_maintenance; in the course scope	Chosen (confirmed)
Docker Compose	Runs the whole stack locally with one command	Chosen (confirmed)
GitHub Actions	Lint and tests on each push	Optional CI

4. Related Architectures and Patterns

- **Cloud reference design:** sensors, IoT Hub or Event Hubs, a stream processor, a data lake and a warehouse, then a BI tool. The project mirrors this with free equivalents, so the design can be moved to the cloud by swapping components.

- **Open-source reference design:** devices, Kafka, Spark or Flink, a lake and a database, then Grafana. This is the design chosen here.
- **Industrial monitoring:** predictive maintenance systems often add machine learning on top of the same pipeline. The project covers the data foundation and leaves models out of scope.

5. Gaps and How the Project Addresses Them

Gap seen in typical tutorials	How the project addresses it
Happy-path data only	Simulator injects late, duplicate and malformed events
No measured results	KPIs for latency, throughput, freshness, errors and uptime, measured and reported
Hard to run elsewhere	One-command Docker Compose setup
Quality checked after the fact	Validation and dead-letter handling inside the stream, plus Great Expectations suites and a daily quality report
Little documentation	Full guideline set: design diagrams, tests, manual, README

6. Summary of Decisions

The confirmed baseline is a Python simulator, Kafka, PySpark Structured Streaming, Apache Parquet for the lake, SQL Server Developer for serving, Great Expectations for data quality, Airflow for orchestration and Docker Compose for local deployment. The dashboard is Grafana. All tools are free to use, fit the course content, and match designs used in industry. Most choices have a cloud equivalent, so the work transfers to Azure.

7. References

Author names, years, venues and page ranges were checked against publisher and index records on 17 Oct 2026. Links were added on 18 Oct 2026 from the publishers' and official project sites.

1. Akidau, T. et al. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. Proceedings of the VLDB Endowment, 8(12), pp. 1792-1803. DOI: 10.14778/2824032.2824076. Page: Google Research.
2. Armbrust, M. et al. (2018). Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. Proceedings of the ACM SIGMOD International Conference on Management of Data, Houston, pp. 601-613. DOI: 10.1145/3183713.3190664.
3. Carbone, P. et al. (2015). Apache Flink: Stream and Batch Processing in a Single Engine. Bulletin of the IEEE Computer Society Technical Committee on Data Engineering, 36(4), pp. 28-38. Record: TU Delft repository. PDF copy: cs.uic.edu.
4. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media, first edition, ISBN 978-1-449-37332-0. O'Reilly page.
5. Kreps, J. (2014). Questioning the Lambda Architecture. O'Reilly Radar, 2 July 2014. Article.
6. Kreps, J., Narkhede, N. and Rao, J. (2011). Kafka: a Distributed Messaging System for Log Processing. NetDB Workshop, Athens, 12 June 2011. PDF copy.
7. Official documentation: Apache Kafka, Apache Spark Structured Streaming Programming Guide, Apache Parquet, Microsoft SQL Server, Great Expectations (GX Core overview), Apache Airflow, Grafana (Microsoft SQL Server data source), Docker Compose.